MINI PROJECT

Student Performance Analytics Using Python

Exploratory Data Analysis and Visualization

Project Overview

This mini project explores how study hours and class attendance relate to student exam performance. It uses Python, Pandas, NumPy, Matplotlib and Seaborn to prepare sample data and create a compact four-chart dashboard.

Objective

To summarize student performance patterns in an easy-to-understand visual format and demonstrate how data analysis can support academic planning.

Tools Used

Python 3 • Pandas • NumPy • Matplotlib • Seaborn

Dataset

The program generates a synthetic dataset of 120 students. It includes study hours, attendance, assignment scores, sleep hours and exam scores. The data is illustrative and should not be treated as real student records.

1. Methodology

1. Generate a reproducible sample dataset using NumPy.
2. Group attendance into bands and study time into ranges.
3. Calculate average exam scores for each group.
4. Plot attendance comparisons, study-hours relationship, score distribution and study-group averages.
5. Save the dashboard as a PNG image.

2. Python Program

Install the required libraries if needed: **pip install pandas numpy matplotlib seaborn**. The complete runnable script is also provided as a separate .py file.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Mini Project: Student Performance Analytics
# Synthetic sample data is generated so the program runs independently.

np.random.seed(42)
n = 120

df = pd.DataFrame({
    "Student_ID": [f"S{i:03d}" for i in range(1, n + 1)],
    "Study_Hours": np.clip(np.random.normal(3.8, 1.5, n), 0.5, 8),
    "Attendance": np.clip(np.random.normal(82, 10, n), 50, 100),
    "Assignment_Score": np.clip(np.random.normal(75, 12, n), 35, 100),
    "Sleep_Hours": np.clip(np.random.normal(6.8, 1.0, n), 4, 9)
})

noise = np.random.normal(0, 8, n)
df["Exam_Score"] = np.clip(
    25 + 5.0 * df["Study_Hours"]
    + 0.25 * df["Attendance"]
    + 0.20 * df["Assignment_Score"]
    + noise, 0, 100
).round(1)

# 1. Average exam score by attendance band
df["Attendance_Band"] = pd.cut(
    df["Attendance"], bins=[0, 65, 80, 90, 101],
    labels=["Below 65%", "65–80%", "80–90%", "Above 90%"]
)
attendance_avg = df.groupby("Attendance_Band", observed=False)["Exam_Score"].mean()

# 2. Study hours vs exam score
# 3. Distribution of exam scores
# 4. Average exam score by study-hours group
df["Study_Group"] = pd.cut(
    df["Study_Hours"], bins=[0, 2, 4, 6, 9],
    labels=["0–2 hrs", "2–4 hrs", "4–6 hrs", "6+ hrs"]
)
study_avg = df.groupby("Study_Group", observed=False)["Exam_Score"].mean()
```

2. Python Program (continued)

```
sns.set_theme(style="whitegrid")
fig, axes = plt.subplots(2, 2, figsize=(13, 9))
fig.suptitle("STUDENT PERFORMANCE ANALYTICS DASHBOARD", fontsize=16, fontweight="bold")

attendance_avg.plot(kind="bar", ax=axes[0, 0], color="steelblue", edgecolor="black")
axes[0, 0].set_title("Average Exam Score by Attendance")
axes[0, 0].set_xlabel("Attendance Band")
axes[0, 0].set_ylabel("Average Exam Score")
axes[0, 0].tick_params(axis="x", rotation=20)

sns.regplot(data=df, x="Study_Hours", y="Exam_Score", ax=axes[0, 1],
            scatter_kws={"alpha": 0.55}, line_kws={"color": "crimson"})
axes[0, 1].set_title("Study Hours vs Exam Score")
axes[0, 1].set_xlabel("Daily Study Hours")
axes[0, 1].set_ylabel("Exam Score")

sns.histplot(df["Exam_Score"], bins=12, kde=True, ax=axes[1, 0], color="seagreen")
axes[1, 0].set_title("Distribution of Exam Scores")
axes[1, 0].set_xlabel("Exam Score")
axes[1, 0].set_ylabel("Number of Students")

study_avg.plot(kind="bar", ax=axes[1, 1], color="darkorange", edgecolor="black")
axes[1, 1].set_title("Average Exam Score by Study Group")
axes[1, 1].set_xlabel("Daily Study Hours")
axes[1, 1].set_ylabel("Average Exam Score")
axes[1, 1].tick_params(axis="x", rotation=0)

plt.tight_layout(rect=[0, 0, 1, 0.94])
plt.savefig("student_performance_dashboard.png", dpi=180, bbox_inches="tight")
plt.show()

print("Average exam score:", round(df["Exam_Score"].mean(), 2))
print("Average attendance:", round(df["Attendance"].mean(), 2), "%")
print("Dashboard saved as student_performance_dashboard.png")
```

3. Output Dashboard

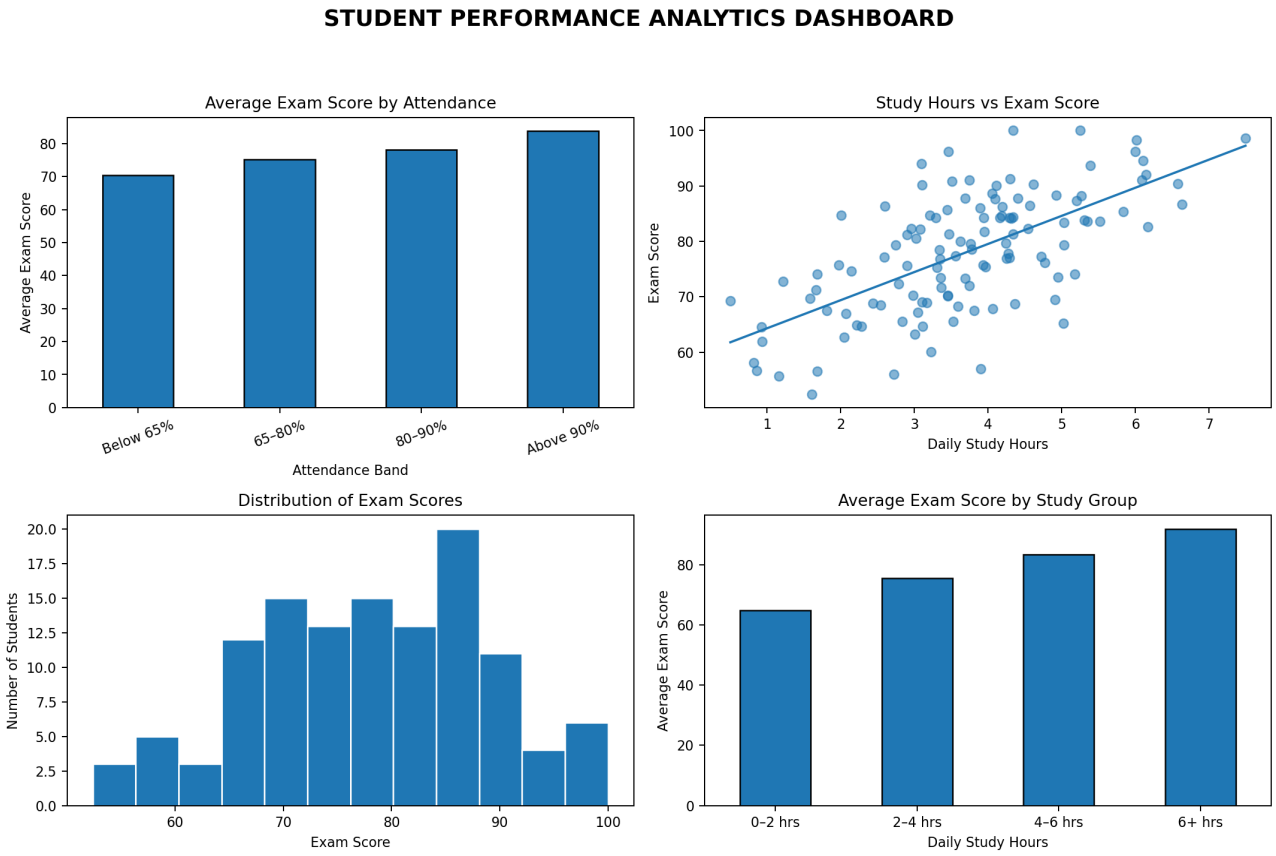


Figure: Four visualizations generated from the synthetic student dataset.

4. Interpretation

- **Attendance comparison:** compares average exam scores across attendance bands.
- **Study-hours relationship:** shows whether higher study time tends to align with higher scores in this sample.
- **Score distribution:** summarizes how exam scores are spread across students.
- **Study-group comparison:** compares average scores across daily study-hour ranges.

5. Conclusion

This project demonstrates a simple end-to-end data analysis workflow: data generation, grouping, summary statistics and visualization. The results can help identify patterns worth investigating, but the synthetic data cannot establish real-world causes or predict an individual student's result.

How to Run

Save the Python script and run **python student_performance_analytics.py** in an environment with the required libraries installed. The dashboard image is saved in the current working directory.